Topic 3: Fourier Transform properties

Linearity, shift, symmetry, convolution, windows, FFT and analysis/synthesis

Digital Signal Processing for Sound and Music · Topic 3

Xavier Serra · Universitat Pompeu Fabra

Lecture roadmap

1. Linearity, shift and symmetry
2. Convolution and filtering intuition
3. Energy, dB and phase unwrapping
4. Zero-padding and zero-phase FFT buffers
5. FFT and analysis/synthesis workflow
6. Example notebook + Exercise 3

Linearity:

Property: $ax_1[n] + bx_2[n] \Leftrightarrow aX_1[k] + bX_2[k]$

$$DFT(ax_1[n] + bx_2[n])$$

$$= \sum_{n=0}^{N-1} (ax_1[n] + bx_2[n]) e^{-j2\pi kn/N}$$

$$= \sum_{n=0}^{N-1} a\, x_1[n] e^{-j2\pi kn/N} + \sum_{n=0}^{N-1} b\, x_2[n] e^{-j2\pi kn/N}$$

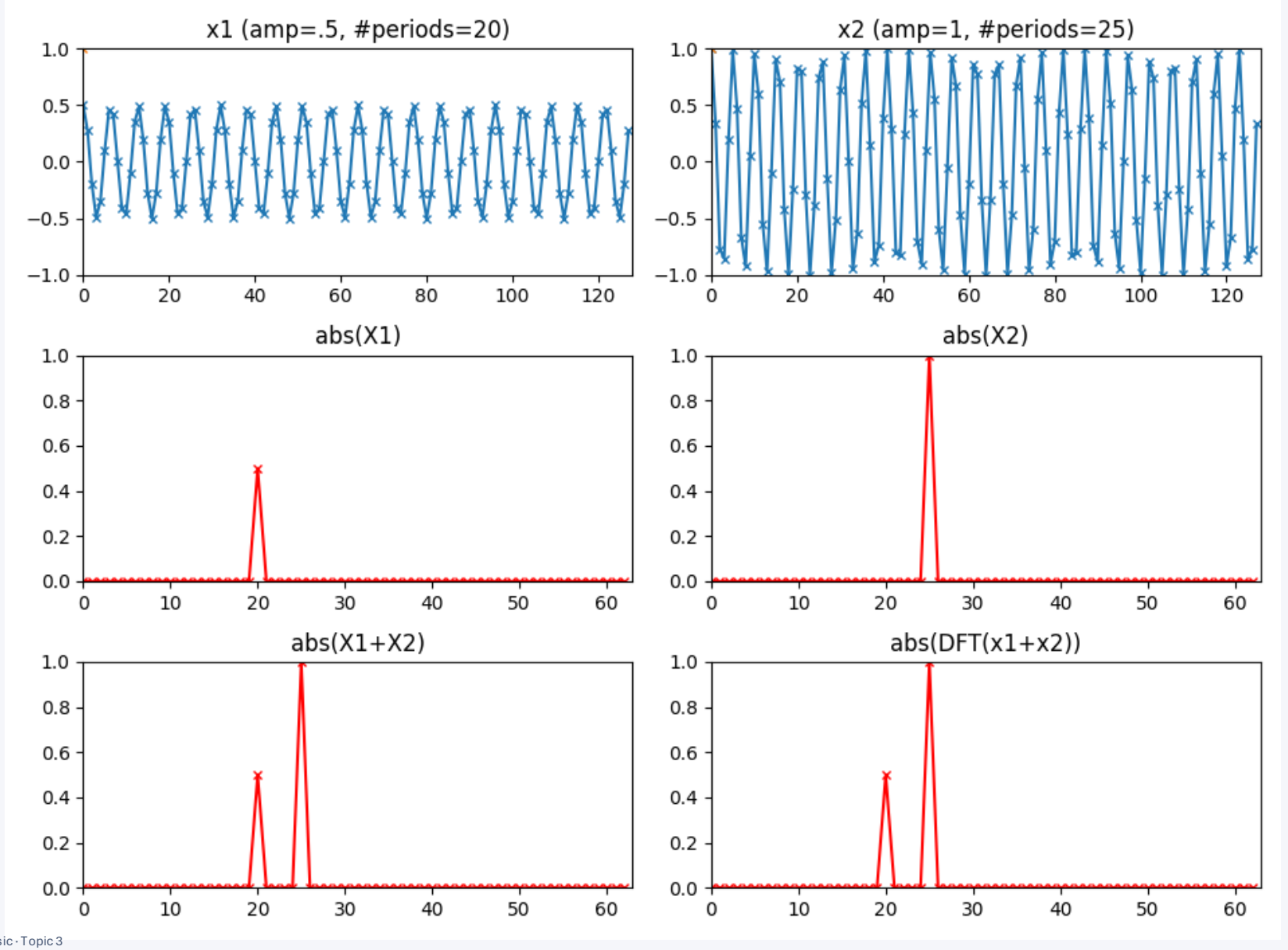
$$= a \sum_{n=0}^{N-1} x_1[n] e^{-j2\pi kn/N} + b \sum_{n=0}^{N-1} x_2[n] e^{-j2\pi kn/N}$$

$$= aX_1[k] + bX_2[k]$$

Linearity

- DFT is a linear transform.
- Analyze sums by summing spectra.
- Useful for additive sound models.

Linearity: spectra add



Shift:

Property: $x[n-n_0] \Leftrightarrow e^{-j2\pi k n_0/N} X[k]$

$$DFT(x[n-n_0])$$

$$= \sum_{n=0}^{N-1} x\,[n-n_0] e^{-j2\pi kn/N}$$

$$= \sum_{m=-n_0}^{N-1-n_0} x\,[m] e^{-j2\pi k(m+n_0)/N} \quad (m = n-n_0)$$

$$= \sum_{m=0}^{N-1} x\,[m] e^{-j2\pi km/N} e^{-j2\pi kn_0/N}$$

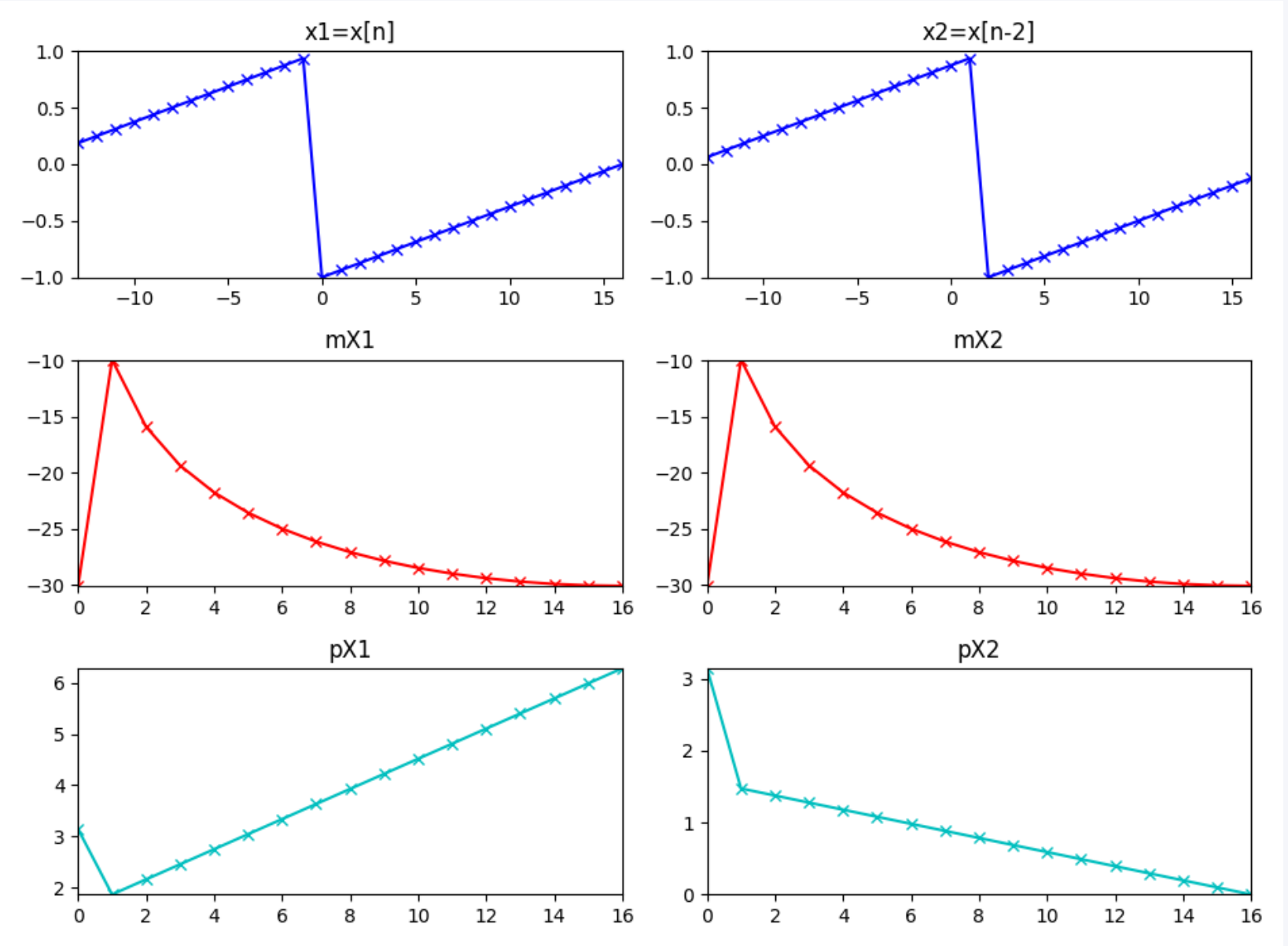
$$= e^{-j2\pi kn_0/N} \sum_{m=0}^{N-1} x\,[m] e^{-j2\pi km/N}$$

$$= e^{-j2\pi kn_0/N} X[k]$$

Shift

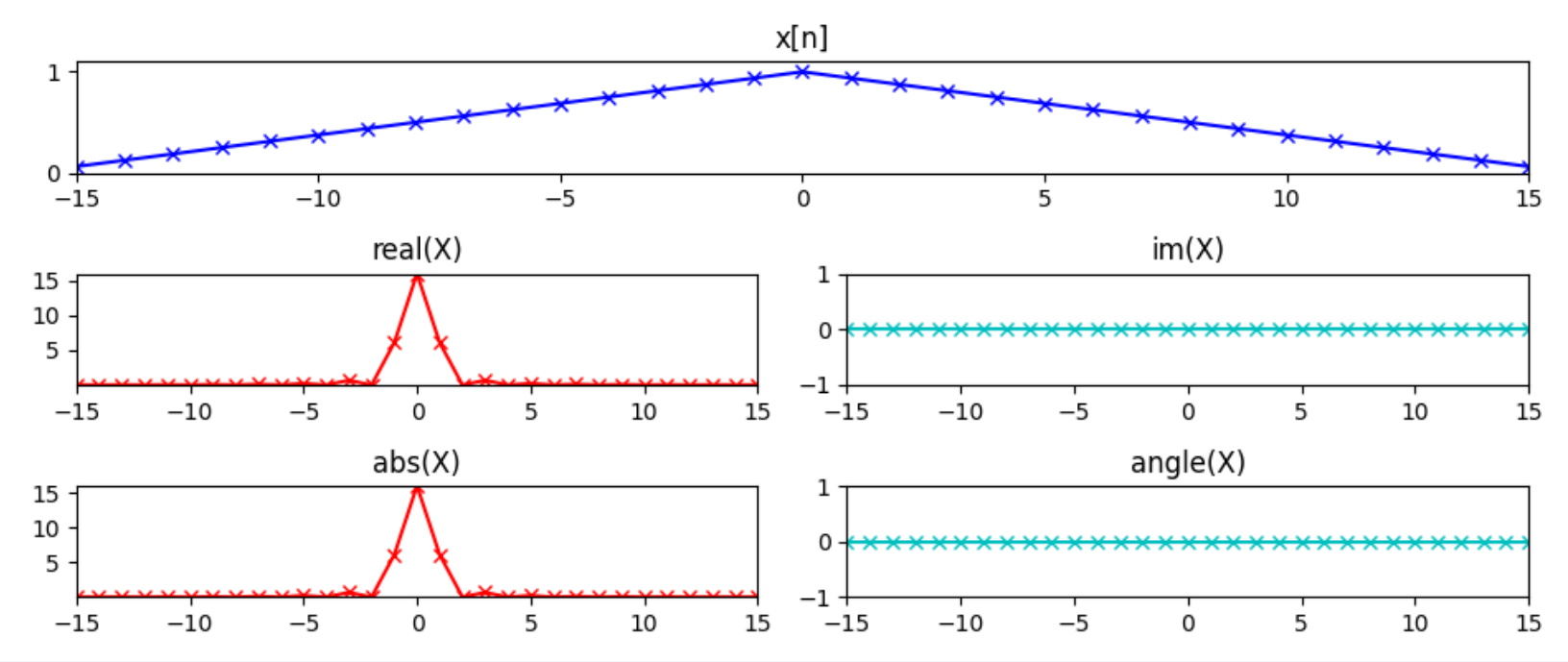
- A time shift does not change magnitude.
- It adds a linear phase term.
- This is why centering matters.

Shift: same magnitude, phase rotation



Symmetry:

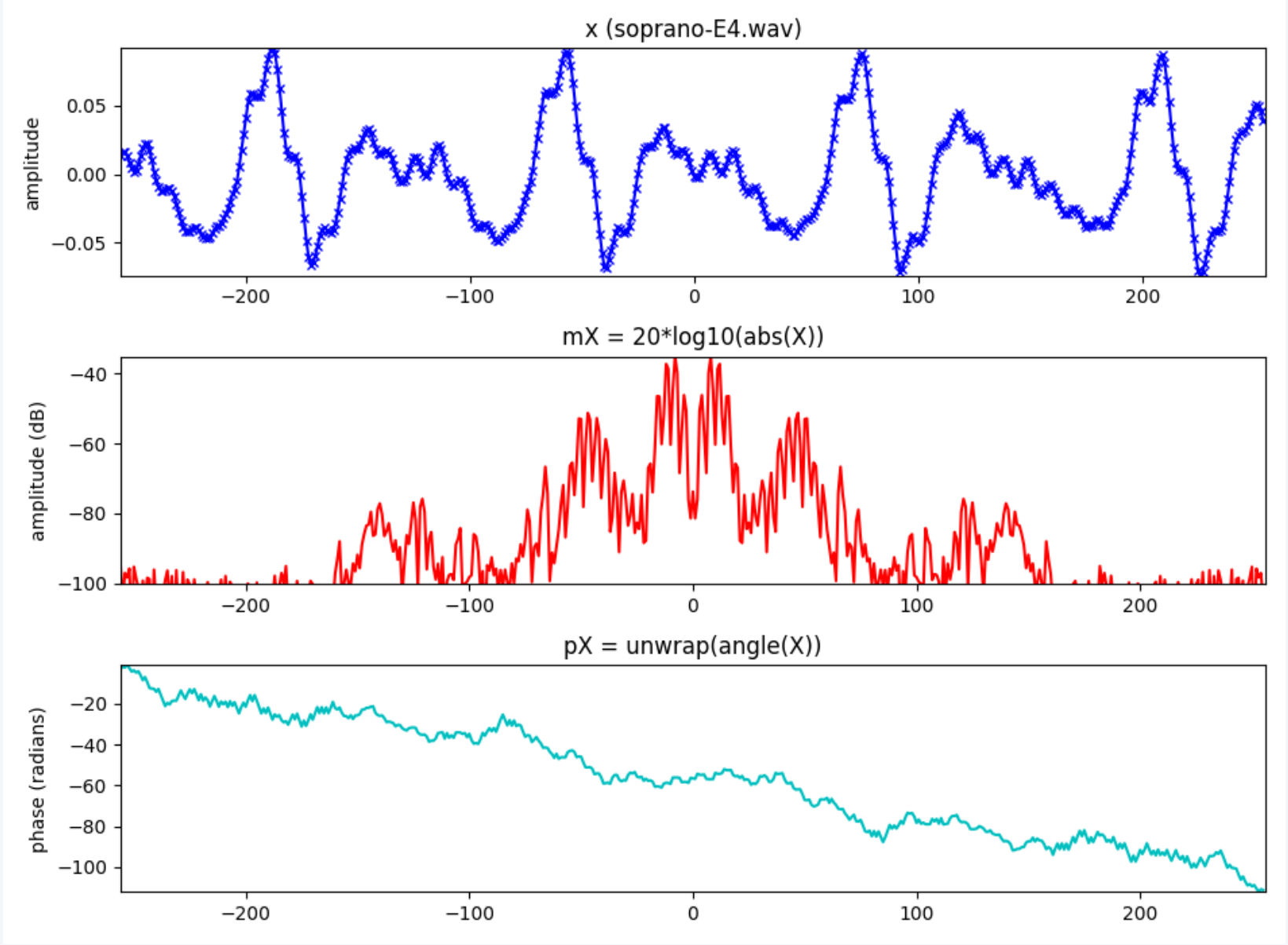
$$x[n] real \Leftrightarrow \Re\{X[k]\} even \text{ and } \Im\{X[k]\}\ odd$$
$$\Leftrightarrow |X[k]| even \text{ and } < X[k] odd$$
$$x[n] real \text{ and } even \Leftrightarrow \Re\{X[k]\}\ even \text{ and } \Im\{X[k]\} = 0$$
$$\Leftrightarrow |X[k]| even \text{ and } < X[k] = n\pi$$



Symmetry

- Real signals have conjugate-symmetric spectra.
- Real + even signals have real spectra.
- These properties reduce storage and computation.

Symmetry for a centered real signal



Convolution:

Property: $x_1[n] * x_2[n] \Leftrightarrow X_1[k] \times X_2[k]$

$$DFT(x_1[n] * x_2[n])$$

$$= \sum_{n=0}^{N-1} (x_1[n] * x_2[n]) e^{-j2\pi kn/N}$$

$$= \sum_{n=0}^{N-1} \sum_{m=0}^{N-1} x_1[m] x_2[n-m] e^{-j2\pi kn/N}$$

$$= \sum_{m=0}^{N-1} x_1[m] \sum_{n=0}^{N-1} x_2[n-m] e^{-j2\pi kn/N}$$

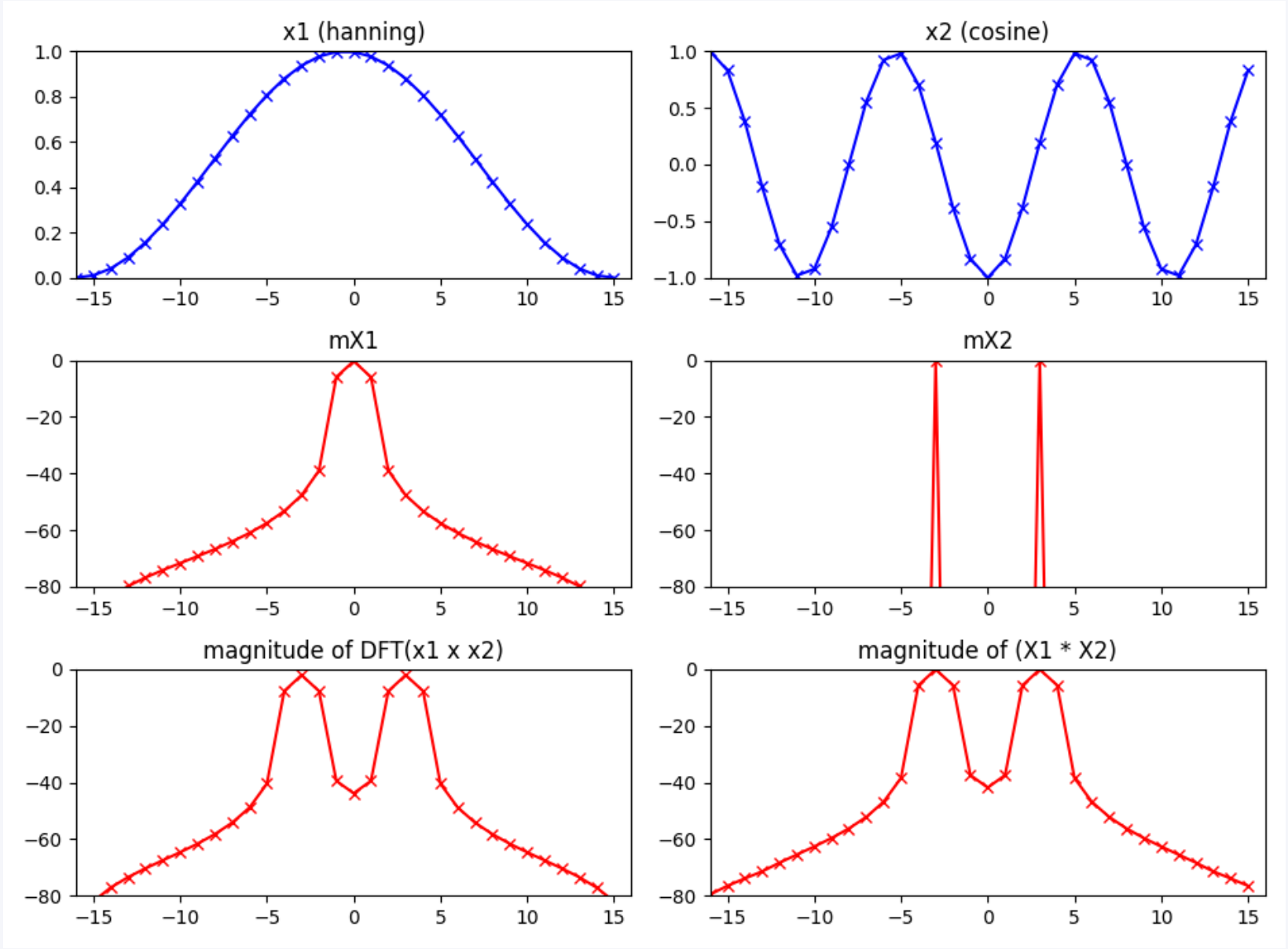
$$= \left(\sum_{m=0}^{N-1} x_1[m] e^{-j2\pi km/N} \right) X_2[k]$$

$$= X_1[k] \times X_2[k]$$

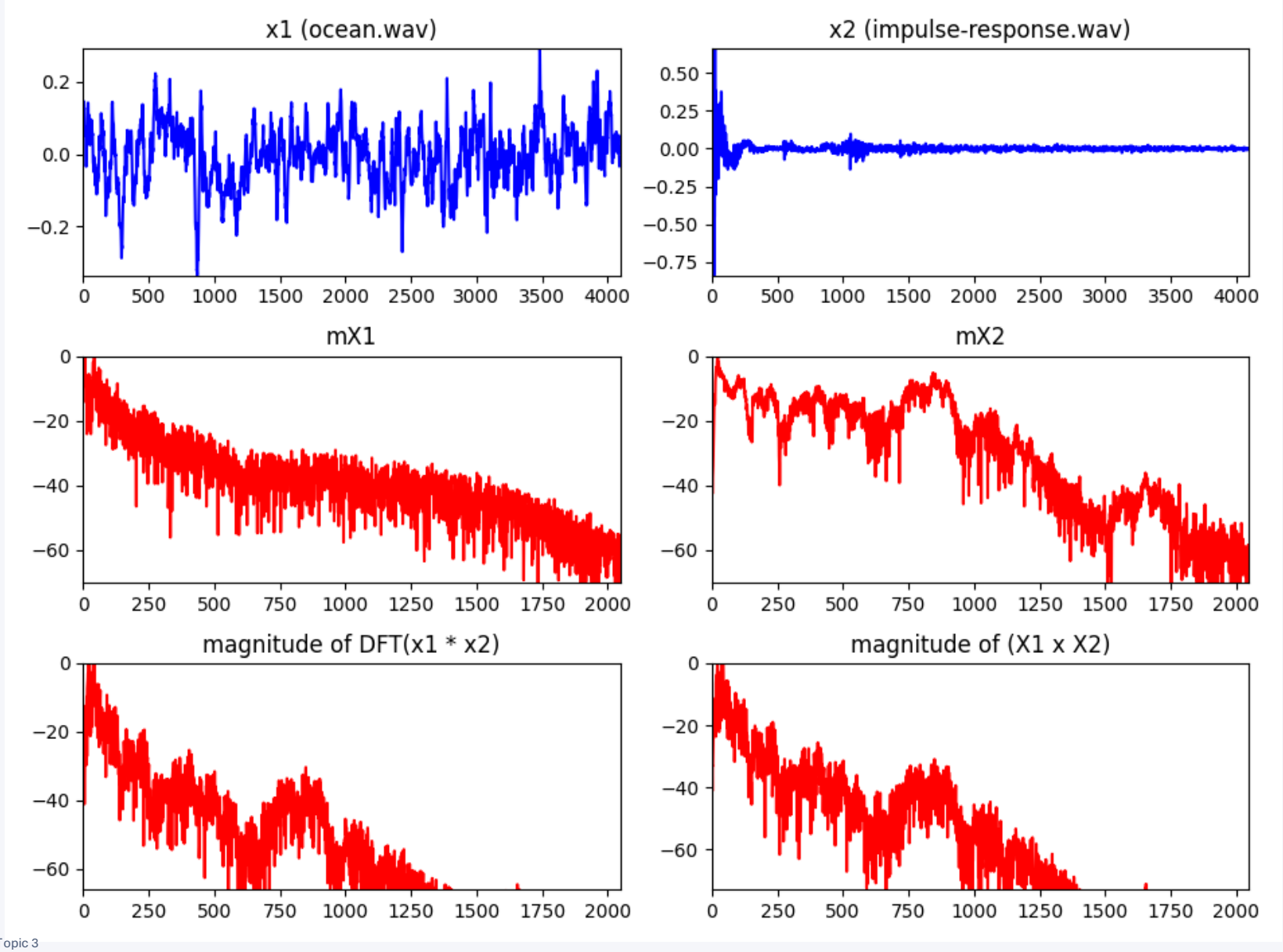
Property:

- Convolution in time becomes multiplication in frequency.
- This is the basis of filtering.
- Circular convolution follows directly from the DFT.

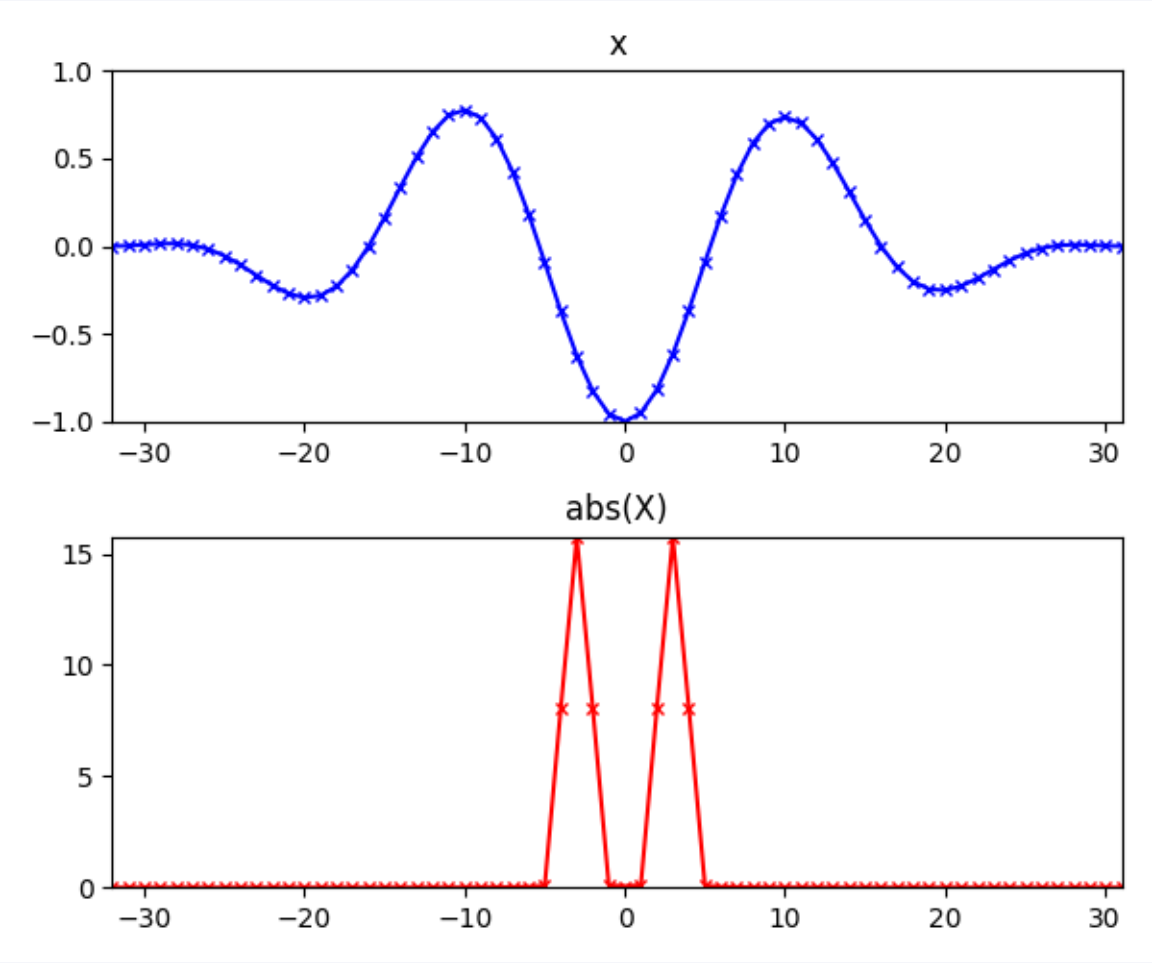
Convolution property



Convolution as frequency-domain filtering



Energy conservation



Parseval identity:

$$\Sigma\,|x[n]|^2 = (1/N)\,\Sigma\,|X[k]|^2$$

time energy:

$$\Sigma\,|x[n]|^2 = 11.81182$$

frequency energy:

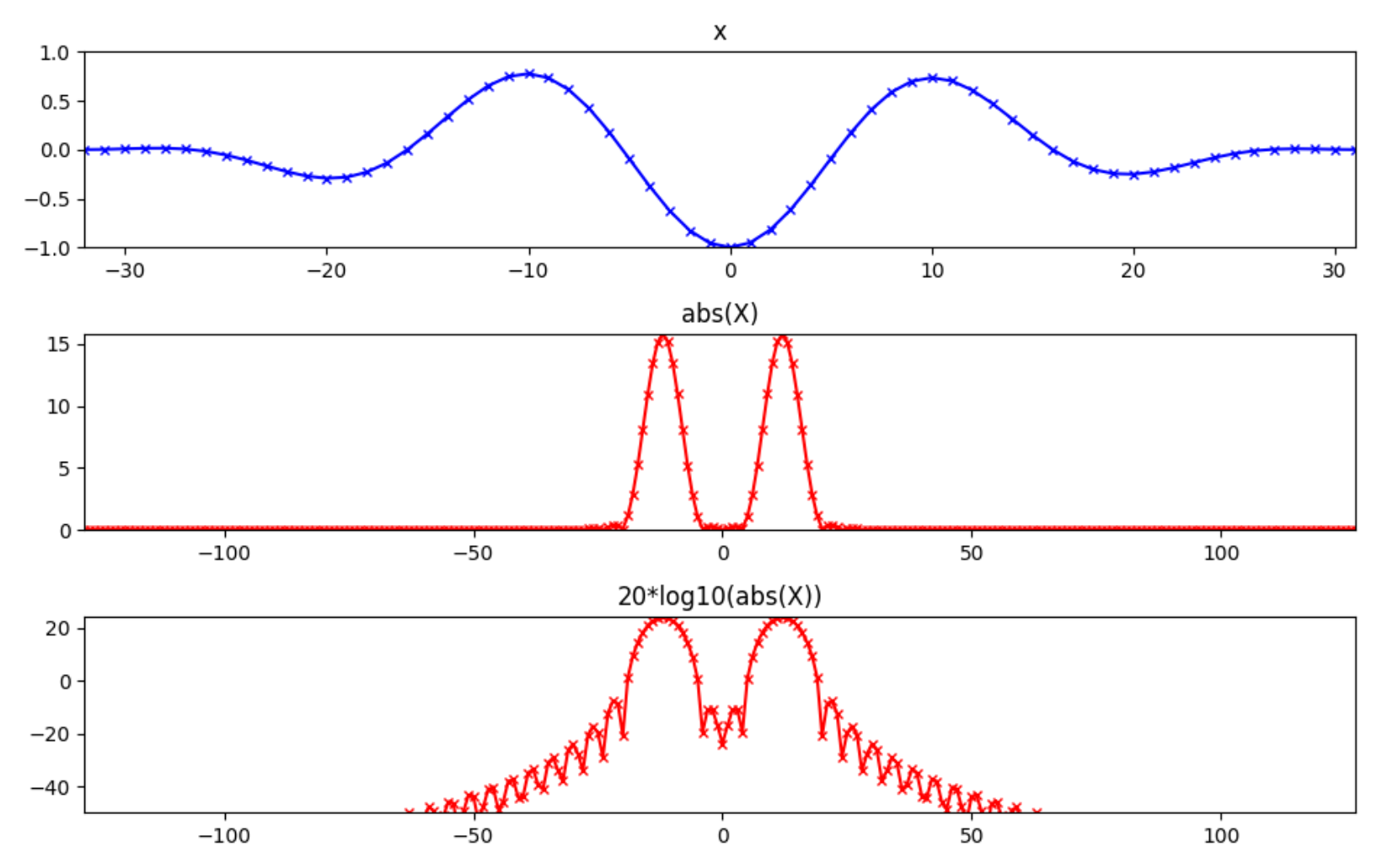
$$(1/N)\,\Sigma\,|X[k]|^2 = 11.81182$$

Parseval

- Energy is conserved across domains.
- Frequency-domain energy has 1/N scaling.
- Useful for checking implementations.

Amplitude in decibels (dB)

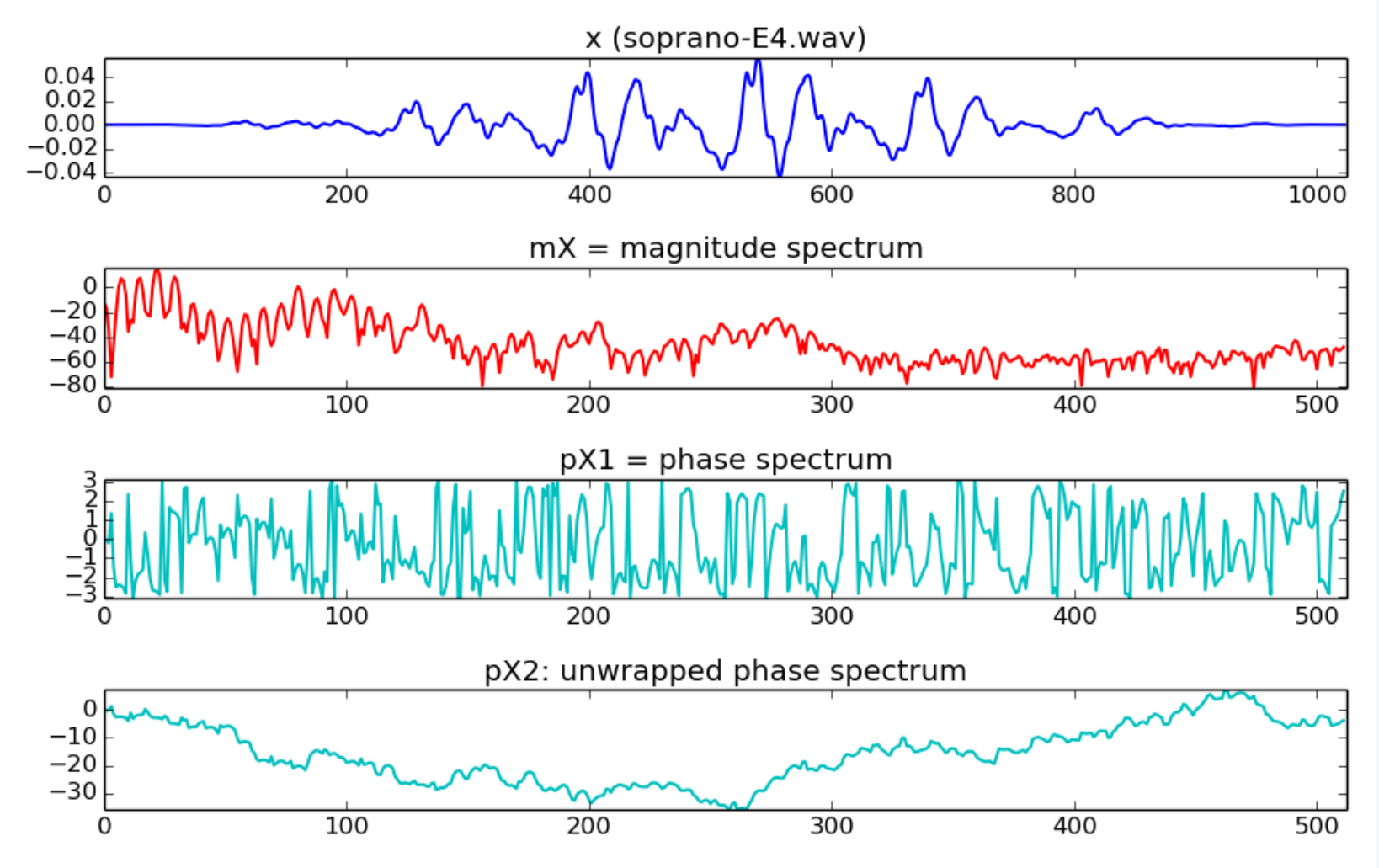
$$X_dB[k] = 20 \log_{10}(\max(|X[k]|, \varepsilon) / ref)$$



dB scale

- 20 log10 maps amplitude ratios to dB.
- Small magnitudes become visible.
- Use a floor before log to avoid −∞.

Phase unwrapping



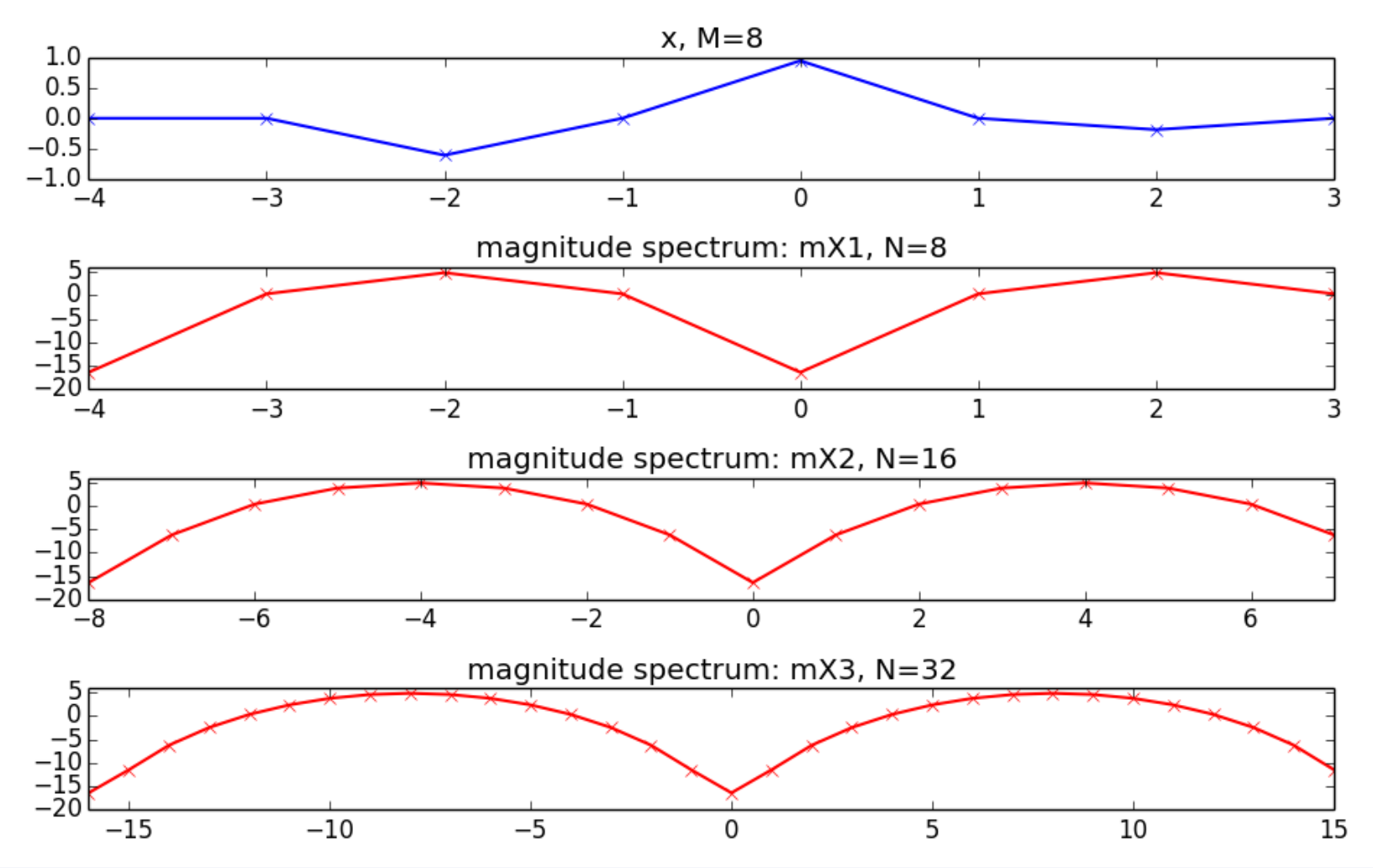
Phase

- Wrapped phase jumps at $\pm\pi$.
- Unwrapping reveals continuous trends.
- Shifts produce phase ramps.

Zero-padding

zero−padding ⇔ interpolation

$$x_zp[n] = x[n] \text{ for } 0 \leq n < M;\ 0 \text{ otherwise}$$



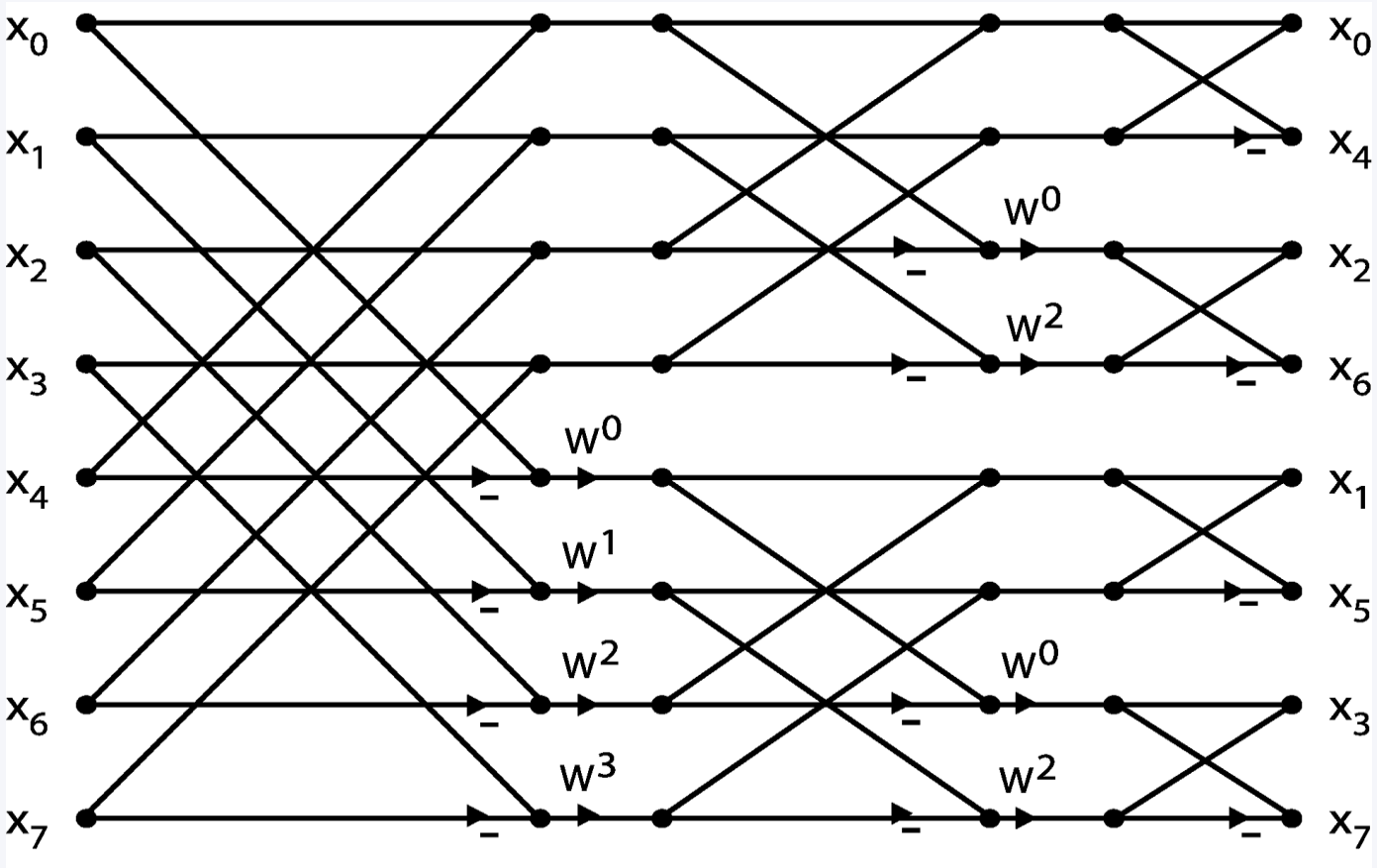
Zero-padding

- Does not add information.
- Interpolates the DFT samples.
- Helps visualize and locate peaks.

Fast Fourier Transform

Cooley-Tukey algorithm: breaks down recursively the DFT of a power of 2 size into two pieces of size N/2.

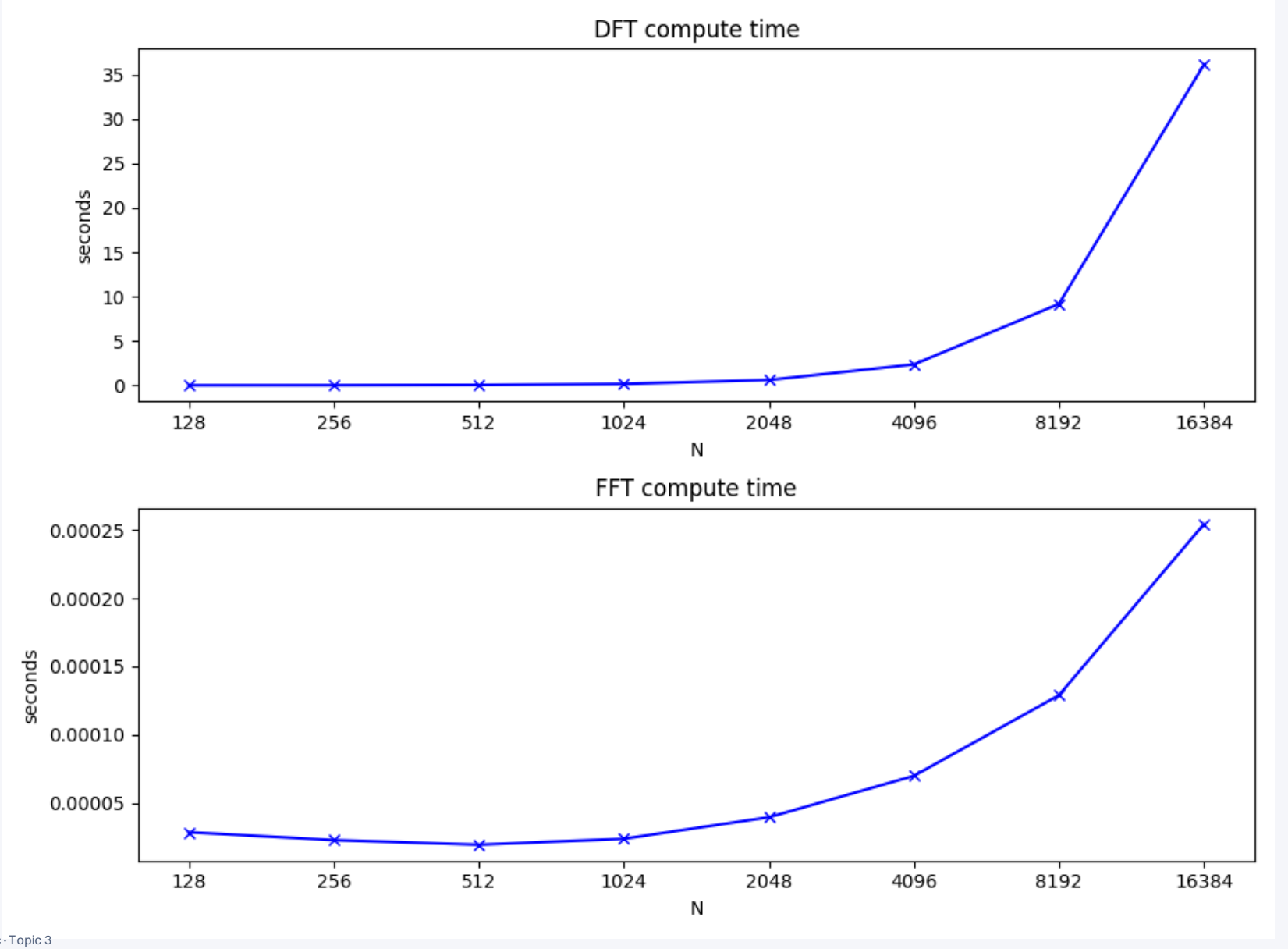
$\text{DFT: } O(N^2) \qquad \text{FFT: } O(N \log_2 N)$



FFT

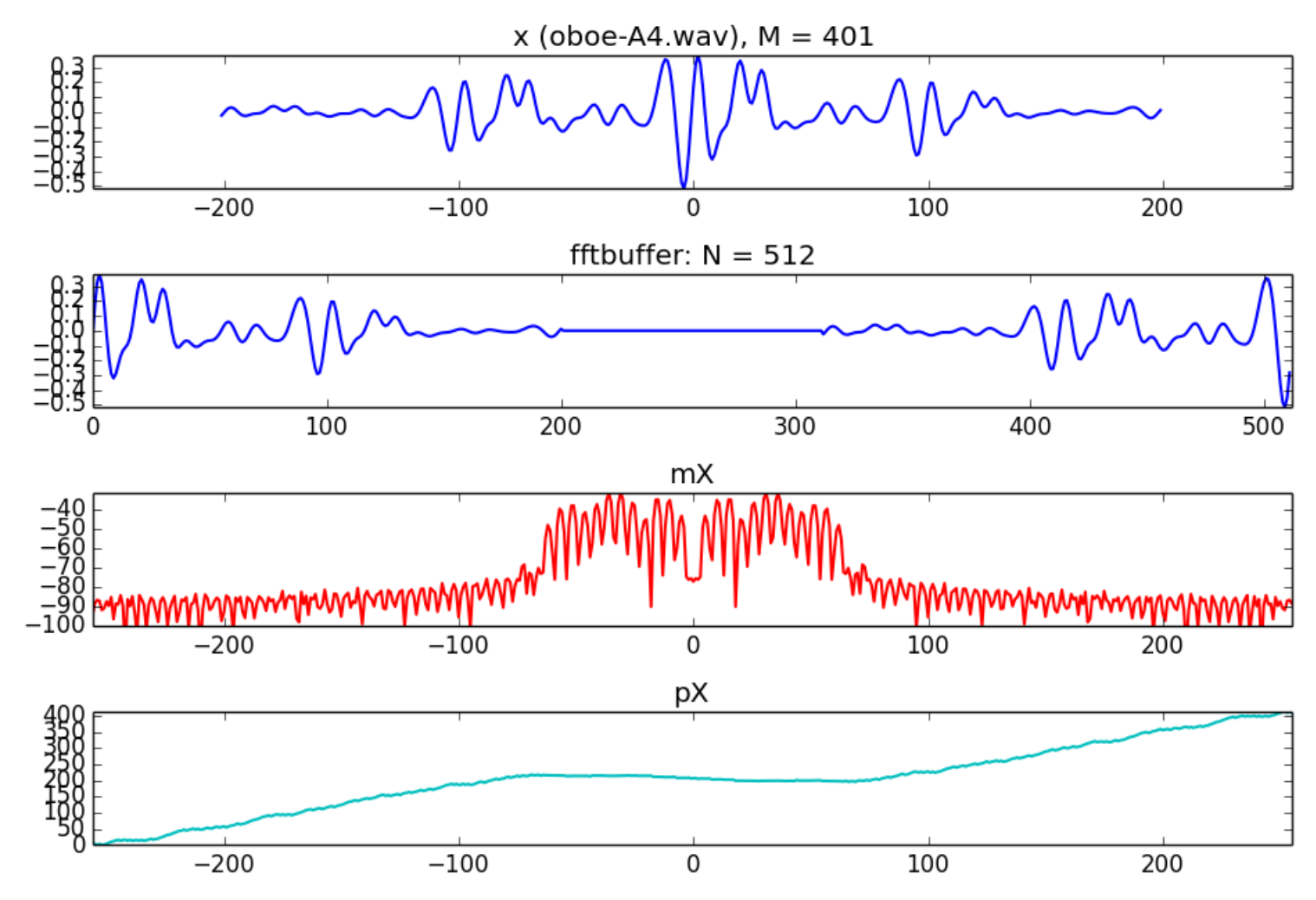
- FFT computes the same DFT.
- It reduces cost from $O(N^2)$ to $O(N \log N)$.
- Power-of-two sizes are often efficient.

DFT vs FFT computational cost



FFT and zero-phase windowing

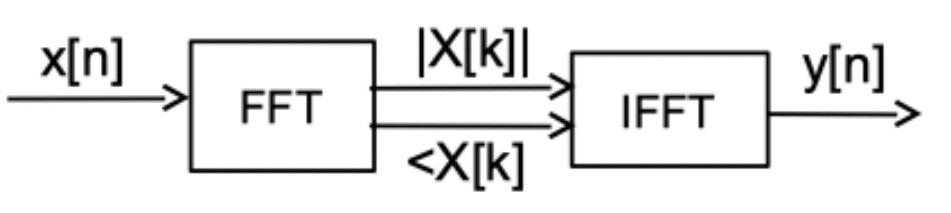
zero-phase FFT: center the frame around n=0 before computing X[k]

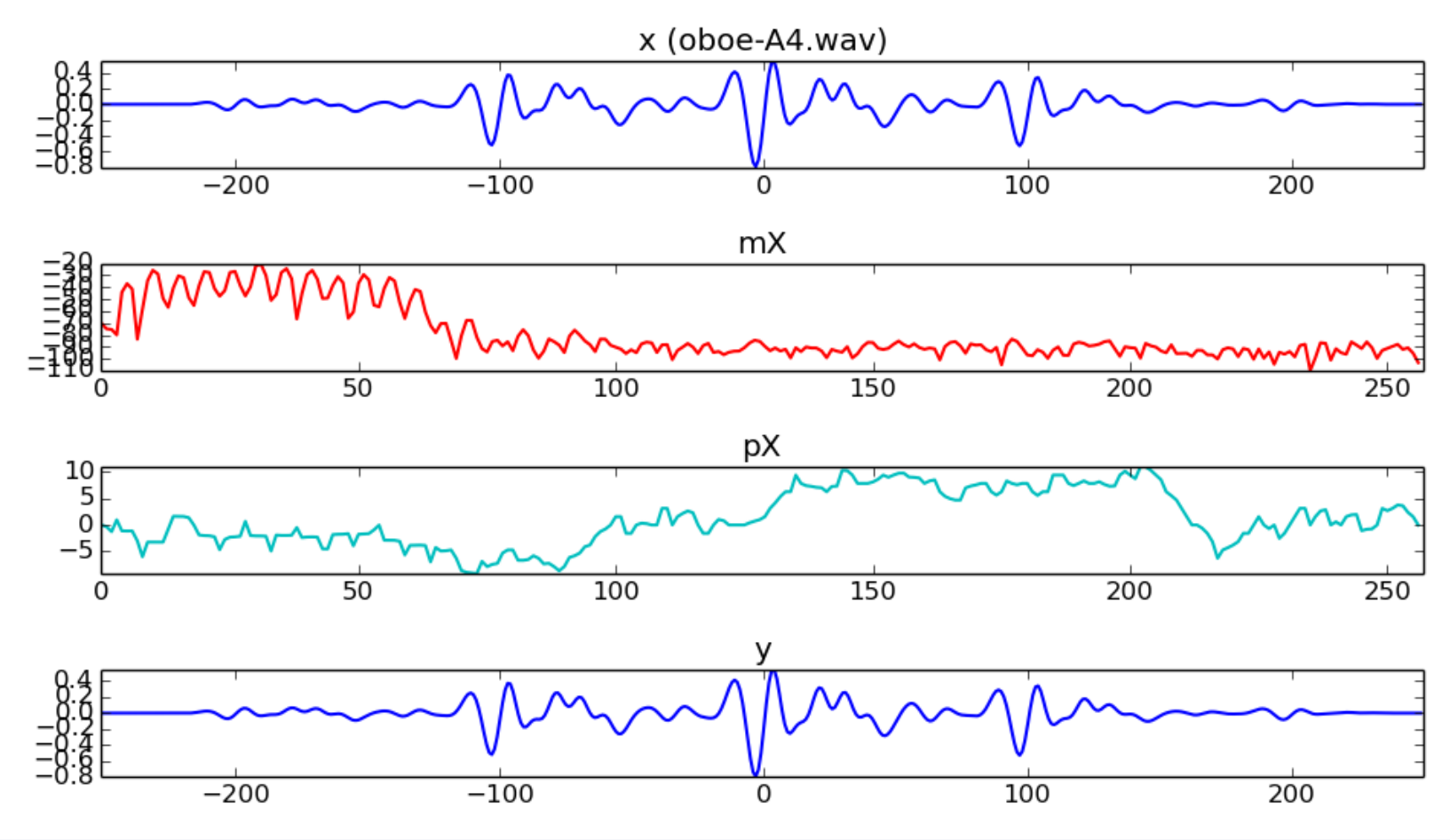


Zero phase

- Center the frame around n = 0.
- Reorder samples into the FFT buffer.
- Avoid an artificial phase ramp.

Analysis/synthesis with magnitude and phase





Analysis / synthesis

- FFT decomposes the frame.
- Magnitude and phase preserve information.
- IFFT reconstructs the signal.

Example notebook: Fourier properties in practice

Use `examples/dft-examples.ipynb` to verify that every spectrum is tied to a precise time-domain operation: framing, windowing, centering, FFT, and inverse FFT.

1 · Frame and center

Select one short sound frame and place it around n = 0 when phase interpretation matters. A non-centered frame introduces a phase ramp.

2 · Window the frame

Apply a Hann/Hamming-type window to reduce boundary discontinuities. Then the spectral shape belongs to the windowed signal, not the raw frame.

3 · Inspect spectrum

Look at magnitude and phase together. For real signals, use conjugate symmetry as a consistency check: magnitudes mirror, phases oppose.

4 · Reconstruct

Use the full complex spectrum for IFFT. Zero-padding can make plots smoother, but it does not add information.

How to read the example outputs

Windowing

Compare the spectrum before and after windowing. A smoother window reduces discontinuities at the frame boundaries and controls spectral leakage.

Symmetry

For real-valued frames, check that positive and negative frequency bins are conjugate pairs. Magnitudes mirror; phases have opposite signs.

Zero-phase / phase

Centering the frame around n = 0 removes the artificial phase ramp caused by a time shift. This makes phase easier to interpret.

Exercise 3: Fourier properties

Third lab: turn Fourier properties into small experiments, plots, checks, and listening tests.

What you will practice

- predict how time operations affect spectra
- compare magnitude, phase, real, and imaginary parts
- use windowing, zero-phase placement, FFT, and IFFT
- separate spectral leakage from zero-padding interpolation

How to start

Open `exercises/E3-Fourier-properties.ipynb`

Use: NumPy · Matplotlib · DFT/FFT · IFFT · plots

Compare with: examples/dft-examples.ipynb

Lab structure

1 · Leakage

- choose M carefully
- compare matched / mismatched frames

2 · Symmetry

- real, even, odd signals
- check expected DFT structure

3 · Filtering

- edit spectral bins
- synthesize and listen

4 · FFT size

- compare M and N
- zero-padding interpolates

5 · Verify

- magnitude + phase
- numerical check + comment

What you will implement in Exercise 3

- Choose M to minimize spectral leakage for two sinusoids using the LCM of periods.
- Test real/even/odd symmetry using zero-phase windowing and inspect real/imaginary parts.
- Manipulate spectral components in the DFT domain and synthesize the filtered signal.
- Compare window size M and FFT size N to separate resolution from zero-padding interpolation.
- Always verify with both magnitude and phase, not magnitude alone.

What to remember from Topic 3

- Fourier properties explain how time-domain operations affect spectra.
- Centering and windowing are essential for interpretable phase.
- Zero-padding interpolates; it does not increase resolution.
- FFT is an efficient DFT implementation.
- Analysis/synthesis requires both magnitude and phase.

References and credits

- Original MTG sms-tools-materials lecture slides and plotting code.
- Example notebook: examples/dft-examples.ipynb
- Exercise notebook: exercises/E3-Fourier-properties.ipynb
- Julius O. Smith, Mathematics of the Discrete Fourier Transform.
- Slides released under CC Attribution-Non Commercial-Share Alike license and code under Affero GPL license.